

17. 上位探索ソルバ

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/upper_solver.py
種別	Python
区分	planner helper
役割	bounded global candidate search、CEM、SHGO、L-BFGS-B を束ねて upper policy を最適化する。
起動文脈	upper planner の数値最適化 backend。

このファイルを読む理由

bounded global candidate search、CEM、SHGO、L-BFGS-B を束ねて upper policy を最適化する。

- hybrid_bounded_minimize が中心。
- global 探索と local refine の接着層である。

呼び出し関係

どこから呼ばれるか

- mpc_node.py
- scripts/solar_sim.py

次に読むと理解が進むファイル

- 特記事項なし。

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
_FORK_COST_FN: Callable[[np.ndarray], float] | None = None

def _fork_finite_cost(vec: np.ndarray) -> float:
    if _FORK_COST_FN is None:
        return float("inf")
    return finite_cost(_FORK_COST_FN, vec)

def clip_to_bounds(vec: np.ndarray, bounds: Bounds) -> np.ndarray:
    out = np.asarray(vec, dtype=float).copy()
    for idx, (lo, hi) in enumerate(bounds):
        out[idx] = float(np.clip(out[idx], lo, hi))
    return out

def finite_cost(cost_fn: Callable[[np.ndarray], float], vec: np.ndarray) -> float:
    try:
        value = float(cost_fn(np.asarray(vec, dtype=float)))
    except Exception:
        return float("inf")
    if not np.isfinite(value):
```

主要構造の読み取り

主要関数は clip_to_bounds, finite_cost, default_seed_library, hybrid_bounded_minimize, add_candidate, local_refine_rows, emit_progress, record_grid_candidate。

主要クラス・関数

行	種別	名前	補足
19	function	_fork_finite_cost	args: vec
25	function	clip_to_bounds	args: vec, bounds
32	function	finite_cost	args: cost_fn, vec
42	function	default_seed_library	args: x0, bounds
57	function	hybrid_bounded_minimize	args: cost_fn, x0, bounds
91	function	add_candidate	args: label, vec
143	function	local_refine_rows	args: pool
201	function	emit_progress	args: payload
225	function	record_grid_candidate	args: idx, candidate_x, candidate_fun
369	function	should_run_cem_from_seed_consensus	args: rows

ファイルを上から読んだときの定義順

行	内容
13	Bounds に Sequence[Tuple[float, float]] の結果を代入する。

16 _FORK_COST_FN に None を代入する。
 19 関数 `_fork_finite_cost` を定義する。
 25 関数 `clip_to_bounds` を定義する。
 32 関数 `finite_cost` を定義する。
 42 関数 `default_seed_library` を定義する。
 57 関数 `hybrid_bounded_minimize` を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や <code>forward reference</code> を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>fromconcurrent. futuresimportProcessPoolExecutor, ThreadPoolExecutor</code>	<code>concurrent.futures</code> から <code>ProcessPoolExecutor</code> , <code>ThreadPoolExecutor</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L273, L280。
4	<code>fromitertoolsimportproduct</code>	<code>itertools</code> から <code>product</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L260。
5	<code>importmultiprocessingasmp</code>	<code>multiprocessing</code> モジュールを利用するため。このファイル内での主な使用位置は L275。
6	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L270。
7	<code>fromtypingimportCallable, Iterable,List,Sequence, Tuple</code>	<code>typing</code> から <code>Callable</code> , <code>Iterable</code> , <code>List</code> , <code>Sequence</code> , <code>Tuple</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L13, L16, L32, L42, L47, L58, L63, L75, ...。
9	<code>importnumpyasnp</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L16, L19, L25, L26, L28, L32, L34, L37, ...。
10	<code>fromscipy. optimizeimportminimize,shgo</code>	<code>scipy.optimize</code> から <code>minimize</code> , <code>shgo</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L148, L311。

関数・クラスを上から順に解説

L19 関数 `_fork_finite_cost`

- 定義: `_fork_finite_cost(vec)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `finite_cost`, `float`
- 戻り値の要点: `finite_cost(_FORK_COST_FN, vec) / float('inf')`

1. 条件 `_FORK_COST_FN is None` を判定し、真なら内部処理を行う。
2. `finite_cost(_FORK_COST_FN, vec)` を返す。

L25 関数 `clip_to_bounds`

- 定義: `clip_to_bounds(vec, bounds)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `clip`, `copy`, `enumerate`, `float`
 - 戻り値の要点: `out`
1. `out` に `np.asarray(vec, dtype=float).copy()` の結果を代入する。
 2. `enumerate(bounds)` を順に走査し、各要素を `(idx, (lo, hi))` に入れて処理する。
 3. `out` を返す。

L32 関数 `finite_cost`

- 定義: `finite_cost(cost_fn, vec)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `cost_fn`, `float`, `isfinite`
 - 戻り値の要点: `value / float('inf') / float('inf')`
1. 例外処理を伴う `try` ブロックを実行する。
 2. 条件 `not np.isfinite(value)` を判定し、真なら内部処理を行う。
 3. `value` を返す。

L42 関数 `default_seed_library`

- 定義: `default_seed_library(x0, bounds)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `append`, `array`, `asarray`, `clip_to_bounds`, `float`, `len`, `linspace`, `maximum`
 - 戻り値の要点: `seeds`
1. `x0` に `np.asarray(x0, dtype=float)` の結果を代入する。
 2. `lo` に `np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)` の結果を代入する。
 3. `hi` に `np.array([float(hi_i) for _, hi_i in bounds], dtype=float)` の結果を代入する。
 4. `span` に `np.maximum(hi - lo, 1e-06)` の結果を代入する。
 5. `seeds` に `[('warm_start', clip_to_bounds(x0, bounds))]` を代入する。
 6. `(0.25, 0.4, 0.55, 0.7, 0.85)` を順に走査し、各要素を `frac` に入れて処理する。
 7. `seeds.append(...)` を実行する。
 8. `seeds.append(...)` を実行する。
 9. `seeds` を返す。

L57 関数 `hybrid_bounded_minimize`

- 定義: `hybrid_bounded_minimize(cost_fn, x0, bounds, maxiter, structured_seeds, cem_enabled, cem_mode, cem_generations, cem_population, cem_elite, local_refine_topk, seed_library_mode, rng_seed, shgo_samples, shgo_iters, cert_grid_levels, cert_grid_values, cert_max_evaluations, cert_workers, progress_callback, cert_progress_interval)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ProcessPoolExecutor`, `ThreadPoolExecutor`, `abs`, `add`, `add_candidate`, `all`, `append`, `array`, `asarray`, `bool`, `clip`, `clip_to_bounds`
- 戻り値の要点: `(np.asarray(best['x'], dtype=float), {'success': bool(best.get('success', True)), 'fun': float(best['fun']), 'label': str(best.get('label', 'best')), 'nit': int(best.get('nit', 0) or 0), 'method': str(best.get('source', 'seed')), 'message': str(best.get('message', '')), 'candidates_evaluated': int(len(evaluated) + discrete_grid_candidates + shgo_nfev), 'cem_mode': cem_mode_norm, 'cem_used': cem_used, 'seed_consensus_spread': consensus_spread, 'shgo_used': shgo_used, 'shgo_nfev': shgo_nfev, 'shgo_local_minima': shgo_local_minima, 'discrete_global_proof': discrete_global_proof, 'discrete_grid_levels': discrete_grid_levels, 'discrete_grid_values': grid_values.tolist() if grid_values.size else [], 'discrete_grid_candidates': discrete_grid_candidates, 'discrete_grid_nonfinite': discrete_grid_nonfinite, 'discrete_grid_best_fun': discrete_grid_best_fun, 'discrete_grid_best_x': discrete_grid_best_x, 'deterministic_seed_candidates': deterministic_seed_candidates, 'deterministic_seed_nonfinite': deterministic_seed_nonfinite, 'finite_library_candidates': int(deterministic_seed_candidates + discrete_grid_candidates), 'seed_library_mode': seed_library_mode_norm, 'selected_x': np.asarray(best['x'], dtype=float).tolist(), 'selected_no_worse_than_grid': selected_no_worse_than_grid, 'finite_library_global_proof': finite_library_global_proof, 'continuous_global_proof': False, 'certificate_scope': certificate_scope}) / (clip_to_bounds(x0, bounds), {'success': False, 'fun': float('inf'), 'label': 'fallback_warm_start', 'nit': 0, 'method': 'fallback', 'candidates_evaluated': int(len(evaluated)), 'cem_used': False}) / refined / (run, spread)`

1. `x0` に `np.asarray(x0, dtype=float)` の結果を代入する。
2. `bounds` に `list(bounds)` の結果を代入する。
3. `lo` に `np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)` の結果を代入する。
4. `hi` に `np.array([float(hi_i) for _, hi_i in bounds], dtype=float)` の結果を代入する。
5. `span` に `np.maximum(hi - lo, 1e-06)` の結果を代入する。
6. `rng` に `np.random.default_rng(int(rng_seed))` の結果を代入する。
7. `evaluated` に `[]` を代入する。
8. `seen` に `set()` を代入する。
9. 関数 `add_candidate` を定義する。
10. `base_seeds` に `list(structured_seeds or [])` の結果を代入する。

処理の流れ

1. ソース中の主要関数を通じて処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/upper_solver.py` を **planner helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。